

1. Introduction

E-Commerce is the process of buying and selling products or services through an online platform. An e-commerce system allows customers to browse products, search for required items, add products to a shopping cart, and place orders electronically.

The purpose of Sprint 1 is to establish the basic architecture and scope of the proposed e-commerce application. It defines the target users, major problems, MVP features, technology stack, and database design.

According to the Sprint 1 requirements, the project focuses on four major areas: **Target Audience & Market Focus, MVP Feature Scope, Tech Stack Selection & Justification, and Entity-Relationship Diagram (ERD).**

2. Target Audience & Market Focus

2.1 Primary Persona

The primary users of our e-commerce system are **retail consumers and small business customers.**

These users need a simple online platform where they can:

- Create an account.
- Login securely.
- Browse available products.
- Search for products.
- Filter products by category.
- Add products to a shopping cart.
- Change product quantities.
- Place orders.
- View their order information.

The system is designed to make online shopping easier, faster, and more convenient.

2.2 Core Pain Point

Traditional shopping can require customers to physically visit stores, search through products manually, and wait for order processing.

The main problems are:

1. Difficulty finding required products.
2. Limited shopping convenience.
3. Manual order processing.
4. Difficulty managing shopping items.
5. Lack of centralized product information.
6. Difficult inventory management for administrators.

Proposed Solution

The proposed e-commerce system provides a centralized online shopping platform. Customers can search and purchase products from anywhere, while administrators can manage products and inventory through the system.

2.3 Domain Scope

The proposed system belongs to the **Consumer Retail / General E-Commerce** domain.

The system covers:

- User management
- Product management
- Category management
- Product searching
- Shopping cart
- Checkout
- Order processing
- Inventory management

The Sprint 1 document specifically requires the project to identify the vertical market/domain served by the application.

3. Minimum Viable Product (MVP)

MVP stands for **Minimum Viable Product**. It represents the minimum set of features required to make the e-commerce application usable.

The project will implement the following major workflows:

Category	Feature	Description	Priority
Authentication	User Registration & Login	Users can create accounts and securely log into the system.	High
Catalog	Product List & Search	Users can browse, search, and filter products.	High
Cart	Cart Management	Users can add, update, and remove products from the cart.	High
Checkout	Order Processing	Users can confirm their order and process payment.	High
Admin	Inventory Control	Admin can add, update, delete, and manage products.	Medium

These feature categories follow the Sprint 1 specification, which requires four to six primary MVP workflows.

4. Description of MVP Features

4.1 User Registration & Authentication

The registration system allows a new customer to create an account using information such as name, email, and password.

The login system verifies the user's credentials before providing access.

Passwords should be stored using secure hashing rather than plain text.

JWT-based authentication can be used for maintaining authenticated sessions.

4.2 Product List & Search

The product catalog allows customers to view available products.

Users can:

- View products.
 - Search products by name.
 - Filter products by category.
 - View product price.
 - Check product availability.
-

4.3 Cart Management

The shopping cart temporarily stores products selected by the customer.

Users can:

- Add products.
 - Increase quantity.
 - Decrease quantity.
 - Remove products.
 - View cart total.
-

4.4 Order Processing

During checkout, the customer confirms the selected products and creates an order.

The system stores:

- Customer ID
- Products
- Quantity
- Unit price
- Total amount
- Order status

- Order date

The assignment allows a mock or Stripe payment gateway for checkout.

4.5 Inventory Control

The administrator manages product inventory.

Admin operations include:

- Add product
- View product
- Update product
- Delete product
- Update stock quantity

These operations are examples of CRUD:

C – Create

R – Read

U – Update

D – Delete

5. Technology Stack

5.1 Frontend – React.js

React.js is selected for developing the user interface.

Reasons:

- Component-based development.
- Reusable UI components.
- Good ecosystem.
- Suitable for interactive shopping interfaces.
- Easy integration with backend APIs.

The Sprint 1 specification identifies React, Next.js, and Vue.js as possible frontend frameworks.

5.2 Backend – Node.js & Express.js

Node.js with Express.js is selected for backend development.

Reasons:

- Fast development of REST APIs.
- Lightweight architecture.
- Large npm ecosystem.
- Good scalability.
- JavaScript can be used on both frontend and backend.

The assignment requires the backend selection to consider performance, scalability, and ecosystem support.

5.3 Database – PostgreSQL

PostgreSQL is selected as the database management system.

Reasons:

- Supports relational database design.
- Strong data integrity.
- Supports primary and foreign keys.
- Supports transactions.
- Suitable for structured e-commerce data.
- Good support for relationships between users, products, orders, and carts.

A relational database is appropriate because the system contains strongly related entities. The Sprint 1 requirements specifically ask for analysis of data integrity and relational versus non-relational design.

5.4 Redis – Optional

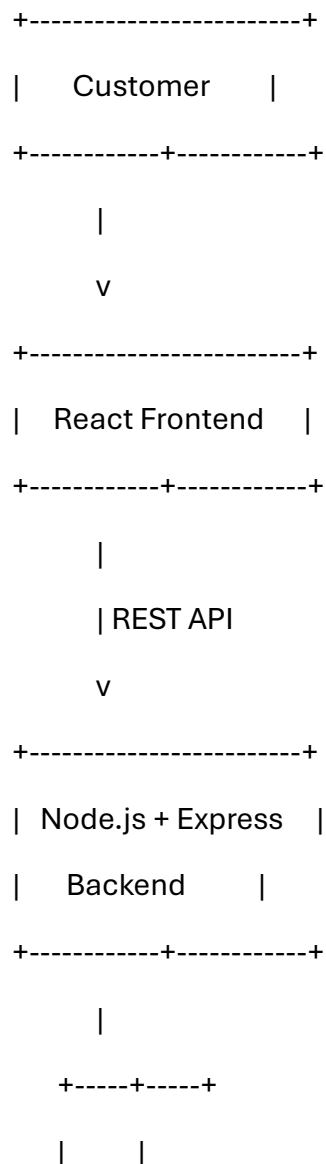
Redis can be used for caching frequently accessed information and temporary session-related data.

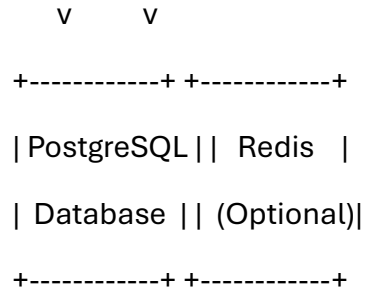
It can improve performance by reducing repeated database queries.

The assignment lists Redis/Celery as optional technologies for caching and asynchronous processing.

6. System Architecture

The basic system architecture is:





Architecture Explanation

The customer interacts with the React frontend. The frontend sends requests to the Node.js/Express backend through REST APIs.

The backend processes business logic and communicates with the PostgreSQL database. Redis can optionally be used for caching and temporary data.

7. Entity-Relationship Diagram (ERD)

The database contains the following major entities:

1. Users
2. Categories
3. Products
4. Orders
5. Order_Items
6. Carts
7. Cart_Items

The Sprint 1 requirements explicitly require Users, Products, Categories, Orders, Order_Items, Cart, and Cart_Items in the relational schema.

7.1 Mermaid ERD Code

erDiagram

```
USERS ||--o{ ORDERS : places
```

```
USERS ||--o| CARTS : owns
```

CATEGORIES ||--o{ PRODUCTS : contains

ORDERS ||--|{ ORDER_ITEMS : contains

PRODUCTS ||--o{ ORDER_ITEMS : included_in

CARTS ||--|{ CART_ITEMS : contains

PRODUCTS ||--o{ CART_ITEMS : added_to

USERS {

INTEGER id PK

VARCHAR email

VARCHAR password_hash

VARCHAR full_name

TIMESTAMP created_at

}

CATEGORIES {

INTEGER id PK

VARCHAR name

VARCHAR description

}

PRODUCTS {

INTEGER id PK

INTEGER category_id FK

```
    VARCHAR name
    VARCHAR description
    DECIMAL price
    INTEGER stock_quantity
    TIMESTAMP created_at
}
```

```
ORDERS {
    INTEGER id PK
    INTEGER user_id FK
    DECIMAL total_amount
    VARCHAR status
    TIMESTAMP created_at
}
```

```
ORDER_ITEMS {
    INTEGER id PK
    INTEGER order_id FK
    INTEGER product_id FK
    INTEGER quantity
    DECIMAL unit_price
}
```

```
CARTS {
    INTEGER id PK
    INTEGER user_id FK
```

```
    TIMESTAMP created_at
    TIMESTAMP updated_at
}
```

```
CART_ITEMS {
    INTEGER id PK
    INTEGER cart_id FK
    INTEGER product_id FK
    INTEGER quantity
}
```

Mermaid.js is specifically recommended in the Sprint 1 document for integrating the ERD directly into Markdown.

8. Database Keys

Primary Keys

A **Primary Key (PK)** uniquely identifies each record.

Entity	Primary Key
Users	id
Categories	id
Products	id
Orders	id
Order_Items	id
Carts	id
Cart_Items	id

Foreign Keys

A **Foreign Key (FK)** connects one table with another table.

Table	Foreign Key References	
Products	category_id	Categories(id)
Orders	user_id	Users(id)
Order_Items	order_id	Orders(id)
Order_Items	product_id	Products(id)
Carts	user_id	Users(id)
Cart_Items	cart_id	Carts(id)
Cart_Items	product_id	Products(id)

The Sprint 1 requirements state that primary keys, foreign keys, cardinality, relationships, and SQL-compliant data types must be explicitly identified.

9. Database Relationships

Users → Orders

1 Relationship

One user can place many orders, but each order belongs to one user.

USERS 1 ----- N ORDERS

Users → Carts

1:1 Relationship

One user has one active shopping cart.

USERS 1 ----- 1 CARTS

Categories → Products

1 Relationship

One category can contain many products.

CATEGORIES 1 ----- N PRODUCTS

Orders → Order_Items

1 Relationship

One order contains multiple order items.

ORDERS 1 ----- N ORDER_ITEMS

Products → Order_Items

1 Relationship

One product can appear in many order items.

PRODUCTS 1 ----- N ORDER_ITEMS

Carts → Cart_Items

1 Relationship

One cart can contain multiple cart items.

CARTS 1 ----- N CART_ITEMS

Products → Cart_Items

1 Relationship

One product can appear in multiple carts.

PRODUCTS 1 ----- N CART_ITEMS

10. SQL Database Coding

```
CREATE TABLE users (  
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    email VARCHAR(255) NOT NULL UNIQUE,  
    password_hash VARCHAR(255) NOT NULL,  
    full_name VARCHAR(100) NOT NULL,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

```
CREATE TABLE categories (  
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name VARCHAR(100) NOT NULL UNIQUE,  
    description VARCHAR(500)  
);
```

```
CREATE TABLE products (  
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    category_id INTEGER NOT NULL,  
    name VARCHAR(200) NOT NULL,  
    description VARCHAR(1000),  
    price DECIMAL(10,2) NOT NULL,  
    stock_quantity INTEGER NOT NULL DEFAULT 0,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
    FOREIGN KEY (category_id)  
        REFERENCES categories(id)  
);
```

```
CREATE TABLE orders (  
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    user_id INTEGER NOT NULL,  
    total_amount DECIMAL(10,2) NOT NULL,  
    status VARCHAR(50) NOT NULL DEFAULT 'Pending',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (user_id)
  REFERENCES users(id)
);
```

```
CREATE TABLE order_items (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id INTEGER NOT NULL,
  product_id INTEGER NOT NULL,
  quantity INTEGER NOT NULL,
  unit_price DECIMAL(10,2) NOT NULL,
```

```
FOREIGN KEY (order_id)
  REFERENCES orders(id),
```

```
FOREIGN KEY (product_id)
  REFERENCES products(id)
);
```

```
CREATE TABLE carts (
  id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id INTEGER NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (user_id)
  REFERENCES users(id)
```

);

```
CREATE TABLE cart_items (  
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    cart_id INTEGER NOT NULL,  
    product_id INTEGER NOT NULL,  
    quantity INTEGER NOT NULL,  
  
    FOREIGN KEY (cart_id)  
        REFERENCES carts(id),  
  
    FOREIGN KEY (product_id)  
        REFERENCES products(id)  
);
```

11. Sample Data

```
INSERT INTO categories (name, description)  
VALUES  
('Electronics', 'Electronic products and accessories'),  
('Clothing', 'Men and women clothing'),  
('Books', 'Educational and general books');  
  
INSERT INTO products  
(category_id, name, description, price, stock_quantity)  
VALUES  
(1, 'Wireless Headphones',
```

'Bluetooth wireless headphones',
4500.00,
20),

(1, 'Smart Watch',
'Digital smart watch',
6500.00,
15),

(2, 'Cotton T-Shirt',
'Comfortable cotton T-shirt',
1800.00,
30),

(3, 'Programming Book',
'Book for learning programming',
2500.00,
10);

12. Example Product Search Query

```
SELECT *  
FROM products  
WHERE name ILIKE '%watch%';
```

Expected Result

Smart Watch

Price: 6500.00

Stock: 15

13. Example Product Listing Query

SELECT

products.id,
products.name,
products.price,
products.stock_quantity,
categories.name AS category

FROM products

JOIN categories

ON products.category_id = categories.id;

Expected Output

ID | Product | Price | Stock | Category

1 | Wireless Headphones | 4500.00 | 20 | Electronics

2 | Smart Watch | 6500.00 | 15 | Electronics

3 | Cotton T-Shirt | 1800.00 | 30 | Clothing

4 | Programming Book | 2500.00 | 10 | Books

14. Project Folder Structure

E-Commerce-Project/

|

|— docs/

| └─ SPRINT_1.md

|

```
├── assets/
|   └── ERD.png
|
├── frontend/
|   └── React Application
|
├── backend/
|   ├── Node.js
|   ├── Express.js
|   └── API
|
├── database/
|   └── schema.sql
|
└── README.md
```

15. GitHub Submission

The required Sprint 1 document should be named:

SPRINT_1.md

It can be placed in the project root or inside the /docs directory.

The GitHub repository should contain the Sprint 1 document and associated diagram assets. The repository URL is then submitted through the LMS.

16. Conclusion

The E-Commerce Sprint 1 assignment establishes the basic foundation of the proposed e-commerce system.

The project defines its target audience, core problem, domain scope, MVP features, technology stack, system architecture, and database structure.

The selected architecture uses React.js for the frontend, Node.js and Express.js for the backend, and PostgreSQL for relational data storage. The ERD defines the relationships between users, products, categories, orders, order items, carts, and cart items.

This architecture provides a clear and practical foundation for developing the complete e-commerce application in the upcoming sprints.

End of Assignment